

```
}
```

OUTPUT

```
Dataset Size: 50000 Searching for target: 49999 (Worst-Case Analysis)
Array Search Result Index: 49999 Time taken by Array Search: 0.000102
seconds Linked List Search Result Index: 49999 Time taken by Linked
List Search: 0.000159 seconds Conclusion: Array search was faster. Why?
Arrays benefit from CPU cache locality (contiguous memory blocks).
```

Experiment-2

Write A Program To Analyze And Compare The Time Complexity Of Basic Operations Insertion On Arrays And Linked Lists Using C

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

#define MAX_SIZE 100000

// Define Linked List Node

Structure typedef struct Node { int data; struct Node* next; }
Node;

// --- ARRAY OPERATIONS ---

// Insert at the beginning of an array: O(N) due to element
shifting

void insertArrayAtBeginning(int arr[], int *size, int value)

{
```

```

if (*size >= MAX_SIZE)

return;

for (int i = *size; i > 0; i--)

{

arr[i] = arr[i - 1]; }

arr[0] = value;

(*size)++;

}

// Insert at the end of an array: O(1) assuming space is
allocated

void insertArrayAtEnd(int arr[], int *size, int value)

{

if (*size >= MAX_SIZE) return;

arr[*size] = value;

(*size)++;

}

// Insert at a specific index in an array: O(N) due to partial
element shifting

void insertArrayAtIndex(int arr[], int *size, int index, int
value)

{

if (*size >= MAX_SIZE || index < 0 || index > *size)

return;

for (int i = *size; i > index; i--)

{

```

```

arr[i] = arr[i - 1];

}

arr[index] = value;

(*size)++;

}

// --- LINKED LIST OPERATIONS ---

// Insert at the beginning of a Linked List: O(1) constant time
Node* insertListAtBeginning(Node* head, int value)

{

    Node* newNode = (Node*)malloc(sizeof(Node));

    newNode->data = value;

    newNode->next = head;

    return newNode; }

// Insert at the end of a Linked List: O(N) due to list traversal
Node* insertListAtEnd(Node* head, int value)

{

    Node* newNode = (Node*)malloc(sizeof(Node));

    newNode->data = value;

    newNode->next = NULL;

    if (head == NULL) return newNode;

    Node* temp = head;

    while (temp->next != NULL)

    {

        temp = temp->next;

```

```

}

temp->next = newNode;

return head;

}

// Insert at a specific index in a Linked List: O(N) due to
traversal to index

Node* insertListAtIndex(Node* head, int index, int value)
{
    if (index == 0)
        return insertListAtBeginning(head, value);

    Node* temp = head;

    for (int i = 0; temp != NULL && i < index - 1; i++)
    {
        temp = temp->next;
    }

    if (temp == NULL)
        return head;

    // Index out of bounds

    Node* newNode = (Node*)malloc(sizeof(Node));

    newNode->data = value;

    newNode->next = temp->next;

    temp->next = newNode;

    return head; } // Helper function to free list memory void
freeList(Node* head)

```

```

{

    Node* temp;

    while (head != NULL)

    {

        temp = head;

        head = head->next;

        free(temp); } }

// --- PERFORMANCE ANALYSIS ---

int main()

{

    int* array = (int*)malloc(MAX_SIZE * sizeof(int));

    int arraySize = 0;

    Node* head = NULL;

    clock_t start, end;

    double timeTaken; /* Seed initial data to both structures up to
MAX_SIZE - 5 */

    for (int i = 0; i < MAX_SIZE - 5; i++)

    {

        array[arraySize++] = i;

        head = insertListAtBeginning(head, i);

    }

    printf("--- Performance Analysis for N = %d Elements ---\n\n",
arraySize);

    // 1. Insertion at the Beginning

```

```

start = clock();

insertArrayAtBeginning(array, &arraySize, 999);

end = clock();

timeTaken = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("Array: Insertion at Beginning Time : %f seconds\n",
timeTaken);

start = clock();

head = insertListAtBeginning(head, 999);

end = clock();

timeTaken = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("Linked List: Insertion at Beginning : %f seconds\n\n",
timeTaken);

// 2. Insertion at the End

start = clock();

insertArrayAtEnd(array, &arraySize, 999);

end = clock(); timeTaken = ((double)(end - start)) /
CLOCKS_PER_SEC; printf("Array: Insertion at End Time : %f
seconds\n", timeTaken); start = clock();

head = insertListAtEnd(head, 999);

end = clock();

timeTaken = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("Linked List: Insertion at End : %f seconds\n\n",
timeTaken);

// 3. Insertion at a Specific Index (Middle)

int targetIndex = arraySize / 2;

start = clock();

```

```

insertArrayAtIndex(array, &arraySize, targetIndex, 999);

end = clock();

timeTaken = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("Array: Insertion at Middle Index : %f seconds\n",
timeTaken);

start = clock();

head = insertListAtIndex(head, targetIndex, 999); end = clock();
timeTaken = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("Linked List: Insertion at Middle : %f seconds\n\n",
timeTaken);

// Cleanup free(array);

freeList(head);

return 0; }

```

OUTPUT

```

Performance Analysis for N = 99995 Elements --- Array: Insertion at
Beginning Time : 0.000215 seconds Linked List: Insertion at Beginning :
0.000001 seconds Array: Insertion at End Time : 0.000001 seconds Linked
List: Insertion at End : 0.000318 seconds Array: Insertion at Middle
Index : 0.000107 seconds Linked List: Insertion at Middle : 0.000164
seconds

```